

LAB 8. THIẾT LẬP LIÊN KẾT GIỮA CÁC BẢNG VÀ HIỂN THỊ DỮ LIỆU LIÊN QUAN

Chủ đề: Liên kết bảng (Relationships) và Tối ưu hóa hiệu năng truy vấn dữ liệu (Eager Loading). Cấu hình mối quan hệ trong Model, hiển thị tên danh mục tương ứng của sản phẩm ngoài giao diện và **kiểm chứng việc tối ưu hóa câu lệnh SQL (giải quyết triệt để N+1 query)** thông qua công cụ log hoặc thanh công cụ kiểm thử.

MỤC TIÊU BÀI LAB

- Biết cách định nghĩa mối quan hệ **Một - Nhiều (One-to-Many)** giữa hai Model Category và Product.
- Hiểu rõ bản chất và tác hại của lỗi **N+1 Query Problem** khi hiển thị dữ liệu liên kết.
- Áp dụng thành thạo kỹ thuật **Eager Loading (with())** để tối ưu hóa tốc độ tải trang.
- Biết cách cấu hình lắng nghe câu lệnh SQL (Query Logging) để chứng minh hiệu quả tối ưu.

THỰC HÀNH

Chúng ta sẽ nâng cấp trang danh sách sản phẩm (products.index) đã làm ở **Lab 07**. Yêu cầu mới là phải hiển thị thêm một cột "**Tên danh mục**" cho từng sản phẩm. Sinh viên phải thực hiện cấu hình liên kết và viết câu lệnh Eloquent đảm bảo chỉ phát sinh tối đa **2 câu lệnh SQL** thay vì bị dính lỗi N+1 query.

BƯỚC 1: THIẾT LẬP LIÊN KẾT TRONG MODEL

Để Laravel hiểu được mối quan hệ giữa hai bảng đã tạo ở Lab 06, ta cần khai báo các hàm quan hệ (Relationship methods) trong hai file Model tương ứng.

1. Trong Model Category (Một danh mục có NHIỀU sản phẩm):

Mở file app/Models/Category.php và thêm phương thức products():

```
namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
```

```
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasMany; // Import class này

class Category extends Model
{
    use HasFactory;

    protected $fillable = ['name', 'slug'];

    /**
     * Khai báo mối quan hệ: 1 Danh mục có nhiều Sản phẩm
     */
    public function products(): HasMany
    {
        // Mặc định Laravel sẽ tự tìm khóa ngoại 'category_id' bên bảng
        products
        return $this->hasMany(Product::class);
    }
}
```

2. Trong Model Product (Một sản phẩm THUỘC VỀ một danh mục):

Mở file app/Models/Product.php và thêm phương thức category():

```
namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\SoftDeletes;
use Illuminate\Database\Eloquent\Relations\BelongsTo; // Import class này

class Product extends Model
{
    use HasFactory, SoftDeletes;

    protected $fillable = ['category_id', 'name', 'price',
    'stock_quantity', 'description'];

    /**
     * Khai báo mối quan hệ ngược: Sản phẩm này thuộc về một Danh mục
     */
    public function category(): BelongsTo
```

```
{
    // Laravel dựa vào tên hàm 'category' để tự suy ra khóa ngoại là
    'category_id'
    return $this->belongsTo(Category::class);
}
```

BƯỚC 2: THỬ NGHIỆM VÀ QUAN SÁT LỖI N+1 QUERY (LAZY LOADING)

1. Sửa tạm thời ProductController theo cách viết thông thường (Chưa tối ưu):

Mở file app/Http/Controllers/ProductController.php, sửa hàm index():

```
public function index()
{
    // CHƯA TỐI ƯU: Chỉ lấy sản phẩm, chưa nạp trước danh mục (Lazy
    Loading)
    $products = Product::latest()->paginate(10);

    return view('products.index', compact('products'));
}
```

2. Cập nhật giao diện View Blade để hiển thị Tên Danh Mục:

Mở file resources/views/products/index.blade.php, thêm cột **Danh mục** vào bảng:

```
<table class="table table-bordered table-striped">
    <thead>
        <tr>
            <th>ID</th>
            <th>Tên sản phẩm</th>
            <th>Danh mục</th> <th>Giá</th>
            <th>Số lượng</th>
            <th>Hành động</th>
        </tr>
    </thead>
    <tbody>
        @foreach($products as $product)
            <tr>
                <td>{{ $product->id }}</td>
                <td>{{ $product->name }}</td>

                <td>{{ $product->category?->name ?? 'Chưa phân loại' }}</td>
```

```
<td>{{ number_format($product->price) }} đ</td>
<td>{{ $product->stock_quantity }}</td>
<td>
    <a href="{{ route('products.edit', $product->id) }}"
class="btn btn-sm btn-info">Sửa</a>
</td>
</tr>
@endforeach
</tbody>
</table>
```

BƯỚC 3: CẤU HÌNH CÔNG CỤ THEO DÕI CÂU LỆNH SQL (QUERY LOGGER)

Để biết được hệ thống đang chạy bao nhiêu câu SQL dưới nền, ta sẽ cấu hình cho Laravel tự động in các câu lệnh SQL ra file log (storage/logs/laravel.log).

Mở file app/Providers/AppServiceProvider.php và cập nhật hàm boot():

```
namespace App\Providers;

use Illuminate\Support\ServiceProvider;
use Illuminate\Pagination\Paginator;
use Illuminate\Support\Facades\DB; // Import Facade DB
use Illuminate\Support\Facades\Log; // Import Facade Log

class AppServiceProvider extends ServiceProvider
{
    public function boot(): void
    {
        Paginator::useBootstrapFive();

        // CẤU HÌNH LẮNG NGHE TRUY VẤN SQL (Chỉ chạy ở môi trường
        local/dev)
        if (config('app.env') === 'local') {
            DB::listen(function ($query) {
                // Ghi câu lệnh SQL kèm thời gian chạy vào file laravel.log
                Log::info("SQL: " . $query->sql . " | Bindings: " .
                json_encode($query->bindings));
            });
        }
    }
}
```

Thực hành kiểm tra lỗi N+1:

1. Mở file log tại đường dẫn storage/logs/laravel.log và xóa sạch nội dung cũ (hoặc xóa file log cũ đi).
2. Lên trình duyệt F5 lại trang danh sách sản phẩm (<http://localhost:8000/products>).
3. Mở lại file laravel.log, sẽ thấy hiện tượng:
 - 1 câu lệnh lấy danh sách sản phẩm: `select * from "products"...`
 - **10 câu lệnh lặp đi lặp lại** để tìm danh mục: `select * from "categories" where "id" = ?...` (Do trang hiển thị 10 sản phẩm).

Tổng cộng phát sinh 11 câu lệnh truy vấn! Đây chính là lỗi N+1 Query.

BƯỚC 4: TỐI ƯU HÓA TRUY VẤN BẰNG EAGER LOADING

Bây giờ, chúng ta sửa lại Controller để áp dụng kỹ thuật **Eager Loading**, hướng dẫn Laravel gộp các câu lệnh tìm danh mục thành một câu duy nhất sử dụng mệnh đề IN.

Mở lại file `app/Http/Controllers/ProductController.php` và cập nhật hàm `index()`:

```
public function index()
{
    /**
     * TỐI ƯU HÓA: Sử dụng phương thức với hàm
     with('tên_hàm_quan_hệ_trong_model')
     * Eloquent sẽ chạy nạp trước toàn bộ danh mục của 10 sản phẩm này ngay
     từ đầu
     */

    $products = Product::with('category')->latest()->paginate(10);

    return view('products.index', compact('products'));
}
```

BƯỚC 5: KIỂM CHỨNG VÀ ĐÁNH GIÁ KẾT QUẢ

1. Tiếp tục xóa sạch nội dung file `storage/logs/laravel.log`.
2. Lên trình duyệt F5 lại trang danh sách sản phẩm lần nữa.
3. Kiểm tra lại file `laravel.log`. Lúc này, hệ thống sẽ **chỉ in ra đúng 2 câu lệnh SQL duy nhất** dù trang web có hiển thị bao nhiêu sản phẩm đi chăng nữa:

Lab 08 - Thiết lập liên kết giữa các bảng và hiển thị dữ liệu liên quan một cách tối ưu

```
-- Câu lệnh 1: Lấy ra 10 sản phẩm mới nhất (để phân trang)
[2026-07-08 16:00:00] local.INFO: SQL: select * from "products" where
"products"."deleted_at" is null order by "created_at" desc limit 10 offset
0

-- Câu lệnh 2: Gom toàn bộ category_id của 10 sản phẩm trên và nạp 1 lần
duy nhất vào RAM
[2026-07-08 16:00:00] local.INFO: SQL: select * from "categories" where
"categories"."id" in (1, 3, 4, 7)
```